

Chapter 7: ORM with Constraints

Gregory S. DeLozier, PhD

In the database-constraints topic we gave the `pets-and-owners` application rules the database enforces. In the object-relational-mappers topic we rebuilt a single pet table with Peewee. This chapter puts the two together. The owner table, the foreign key, and the rules about valid values all move into Peewee models, and the routes keep calling the same database functions they called before.

The interesting question is where each rule lives now. We will handle a bad request at every layer we can, and each layer has its own job.

Layer	What it does for the application
Browser	Answers quickly, so a person learns about a mistake before anything is sent
Web application	Makes the movement of data reliable, and turns a failure into a response the person can use
Database layer	Guarantees that stored data follows the rules, no matter what happens above it. It has two enforcement points: the Python checks in <code>database.py</code> , and the rules in the tables themselves

A check in the browser can be skipped by anyone who sends a request directly, so the web application checks again. A check in the web application can be skipped by any other code that calls the database functions, so the database functions check again. Code that skips those functions and talks to Peewee, or to SQLite directly, can skip them too. The last line is the table itself, which refuses a bad row whatever code tried to save it. Following one bad request down through those layers, and through the two enforcement points in the database layer, is most of the chapter.

Use the files in `chapter-07-orm-with-constraints/code`. This example has its own `pets.db`, and the first thing we do is make sure your database has the tables the code expects.

Starting with a Current Database

The models in this chapter describe tables with rules built in: `CHECK` constraints, a required owner, a foreign key that restricts deletion. Peewee creates those tables when they are missing. It does not change a table that already exists.

(1) A `pets.db` left over from an earlier example keeps whatever definition it had, and that causes two kinds of trouble.

- **An older table keeps its old rules.** Suppose the file came from the database-constraints topic. It has an owner table and a pet table with `owner_id`, so the application would run against it. But those tables have no `CHECK` rules. On such a database the direct SQL `insert into pet (name, type, age, owner_id) values (' ', 'dog', -4, 1)` succeeds. The Python checks in this chapter would still refuse the same values, but the table would not, and the table is the layer that is supposed to hold no matter what.
- **An older table may not fit at all.** The pet table from the object-relational-mappers topic has no owner link, so there is nowhere to put an owner reference. The one from the database-abstraction topic keeps the owner's name as text.

Deleting the old tables and starting over would fix both problems and would throw the data away. Instead, this example builds new tables and reloads the data into them. The command is:

```
python3 upgrade_database.py
```

It takes an optional filename, like the setup command. What it does depends on the file:

The file	What happens
Does not exist	Empty, current tables are created
Already has the rules the check looks for	Nothing changes
Has an older layout	New tables are built, the data is reloaded, and the new file replaces the old one

For an older layout, the script builds the new tables in a separate file beside the original, reloads every old row through the models, and then swaps the new file into place. The original stays next to it as `pets.db.before-upgrade`, so nothing is dropped, and a failed upgrade leaves the original in place.

The application does not run this for you. When `initialize` finds one of the older layouts, it raises `SchemaNotCurrent` and names the fix. Rebuilding tables quietly inside a normal startup would bring the data-loss problem back.

Once we have seen the models, a later section shows how the rebuild works, what it reports, and how modest the check behind this decision is.

Running the Example

From a working copy of the code directory:

```
python3 -m pip install -r requirements.txt
python3 upgrade_database.py
flask run
```

The supplied `pets.db` has the current tables, so the upgrade reports that and changes nothing. Open the pet list at:

```
http://127.0.0.1:5000/list
```

The application prepares the tables when it starts if they are missing, and `python3 setup_database.py` does the same without running Flask. As in the earlier chapters, create an owner before creating a pet. The pet form has nothing to offer in its owner dropdown until an owner exists.

The tests use temporary files:

```
python3 -m unittest -v
```

The module also has short embedded tests, run with `python3 database.py`. They recreate `test_pets.db` and leave `pets.db` alone.

The Models Carry the Rules

In the database-constraints topic the rules were written in SQL and the application created the tables by executing that SQL. Here the models describe the tables, and Peewee writes the SQL. Both models inherit from a base class that binds them to a database object, as we did before. Here is the owner:

```
class Owner(BaseModel):
    name = TextField(null=False, constraints=[Check("length(trim(name)) > 0")])
    city = TextField(null=True)
    type_of_home = TextField(null=True)
```

`null=False` becomes `NOT NULL`. The new piece is `constraints`, which takes a list of extra SQL clauses to attach to the column. `Check` builds a `CHECK` constraint from the expression you give it. (1)(2)

A **CHECK constraint** is a rule written as an expression. SQLite evaluates it for every row inserted or updated, and the change fails if the expression is false. (3)(4) This one says that a name with all the spaces trimmed off must have at least one character. `NOT NULL` alone would accept `' '` or a name made of spaces, because both are values.

The pet model adds two more checks and the link to its owner:

```
class Pet(BaseModel):
    name = TextField(null=False, constraints=[Check("length(trim(name)) > 0")])
    type = TextField(null=False, constraints=[Check("length(trim(type)) > 0")])
    age = IntegerField(default=0, constraints=[Check("age >= 0")])
    owner = ForeignKeyField(Owner, backref="pets", null=False, on_delete="RESTRICT")
```

Two things are missing compared with the earlier owner-and-pet example: the `food` column and the explicit `AUTOINCREMENT`. The food exercise from that example is not part of this one, and Peewee's default integer key does not promise that identifiers are never reused after a delete.

Peewee turns the pet model into this table definition. (2)

```
CREATE TABLE "pet" (
    "id" INTEGER NOT NULL PRIMARY KEY,
    "name" TEXT NOT NULL CHECK (length(trim(name)) > 0),
    "type" TEXT NOT NULL CHECK (length(trim(type)) > 0),
    "age" INTEGER NOT NULL CHECK (age >= 0),
    "owner_id" INTEGER NOT NULL,
    FOREIGN KEY ("owner_id") REFERENCES "owner" ("id") ON DELETE RESTRICT
)
```

You can read that definition from a running database with `sqlite3 pets.db .schema`. It is the same idea as the hand-written table we used before, and the rules sit in the SQL, where SQLite enforces them. Notice that `age` came out `NOT NULL`. The model says `default=0` without `null=True`, so Peewee treats the column as required.

Rebuilding an Older Database

Now we know which tables the models describe, so we can look at how an older file becomes one of them. The script reads the old rows and inserts each one through the models, so every row has to pass the same rules as a new record. Identifiers are preserved, which keeps the edit and delete links meaningful. If anything goes wrong, the original file is still there.



Reloading rows can go wrong in ways worth knowing about, and the script reports each one instead of hiding it:

- **A row that breaks a rule is skipped.** A pet with age `-4` or a blank name cannot be inserted into a table that forbids both. The report lists it with the reason. The row is still in the backup file.
- **A pet with no owner needs one.** The pet table from the object-relational-mappers topic never recorded owners. Those pets are attached to a new owner named `Unassigned`, and the report says how many. Where an older table stored the owner's name as text, the script creates one owner per distinct name.
- **A column with nowhere to go is left behind.** This chapter's pet table has no `food` column, so `food` values are not carried over. The report names the column, and the backup keeps the values.

Here is what the report looks like for a database from the object-relational-mappers topic:

```
Upgraded pets.db.  
found: There is a pet table but no owner table.  
found: The pet table has no owner_id column.  
found: The pet table has no CHECK rules.  
loaded 1 owners and 2 pets  
2 pets had no owner and now belong to 'Unassigned'  
not carried over (no column for them here): food  
original file kept as pets.db.before-upgrade
```

The check that decides whether a file needs rebuilding is deliberately modest. `schema_problems` looks for the differences between the layouts earlier examples produced: a missing owner table, no `owner_id` column, no `ON DELETE RESTRICT`, and no `CHECK` rules. It does not compare every rule and index in the models, so a file that passes it is not proven identical to what Peewee would build today. If you change the models later, for example by adding an index, files made before the change will still pass. Keeping an explicit schema version in the database, such as SQLite's `PRAGMA user_version`, is the usual way to make that decision reliable, and it is a reasonable next step for this example.

A Foreign Key Is a Field

`ForeignKeyField(Owner, ...)` declares the relationship. Its arguments map onto the pieces of the SQL version. (2)(5)

Argument	What it does
<code>Owner</code>	The model whose primary key this column refers to
<code>null=False</code>	Every pet must have an owner (NOT NULL)
<code>on_delete="RESTRICT"</code>	An owner who still has pets cannot be deleted
<code>backref="pets"</code>	Adds a <code>pets</code> attribute to each owner instance

The field is named `owner`, and Peewee stores the identifier in a column called `owner_id`. On a pet instance, `pet.owner_id` is the plain integer and `pet.owner` is the whole `Owner` object. Peewee looks that object up when you first ask for it. The reverse direction, `owner.pets`, is the `backref`: a query for all the pets that point at that owner.

The same two rules apply to `owner_id` as before. NOT NULL requires a value, and the foreign key requires that the value name an existing owner. (6) Keeping references valid like this is called **referential integrity**. (7) Peewee also creates an index on `owner_id`, so finding one owner's pets does not scan the whole table.

Turning Foreign Keys On

SQLite only enforces foreign keys on a connection that has asked for it. When we first met foreign keys, the setup code executed `PRAGMA foreign_keys = ON` after connecting. Here the setting goes in when the database object is initialized: (6)(8)

```
def initialize(database_file):
    if not db.is_closed():
        db.close()

    problems = schema_problems(database_file)
    if problems:
        raise SchemaNotCurrent(...)

    db.init(database_file, pragmas={"foreign_keys": 1})
    db.connect(reuse_if_open=True)
    db.create_tables([Owner, Pet])

    fk = db.execute_sql("PRAGMA foreign_keys").fetchone()[0]
    assert fk == 1, "Foreign key constraints are not active on this connection."
```

The pragmas dictionary is applied to each new connection, and the assertion reads the setting back so a misconfiguration fails at startup instead of quietly allowing bad references. The `/health` route runs the same check. The `schema_problems` call is the guard described in the section on rebuilding an older database. It opens the file read-only, so looking never changes anything.

As before, the module name `database` and the object `db` have different jobs. `database.initialize()` is our function. `db` is the Peewee object the module uses to manage the connection, and the routes reach it as `database.db` when they need it.

Three Layers, Four Checks

Take a pet with age `-3` and follow it toward the table.

1. **Browser.** The age field is `<input type="number" min="0">`, and the name, type, and owner fields are required. The browser refuses to submit the form and points at the field. Nothing reaches the server.
2. **Web application.** Suppose the request arrives anyway, from a script or from someone who removed the attributes in the browser's developer tools. The route runs `check_pet_form` before it calls the database layer, and answers `400` with a message: `Error: age must be a whole number, zero or more.`
3. **Database layer: the Python checks.** Suppose some other code calls `create_pet` directly. `_normalize_age` raises `ValueError("Age must be non-negative.")` before any SQL runs.
4. **Database layer: the table.** Suppose code skips those functions and calls `Pet.create` or runs SQL directly. `CHECK (age >= 0)` makes SQLite refuse the insert.

Each layer does something the others cannot. The browser is the only one that can answer while the person is still typing, and it is the only one a person can bypass without effort. The web application is where a request becomes data, so it is where malformed or missing fields get caught and explained. The database layer is where the data lives. Its Python checks protect every caller of the database functions, and its table rules are the only enforcement that sees every write, from any source. When we say the data follows the rules, the table is what we mean.

A table rule is only as strict as it is written. `CHECK (age >= 0)` does not stop the text `abc` when it arrives through direct SQL, because SQLite treats any integer as less than any text value, so the comparison passes. (9) The Python check and Peewee's integer field catch it earlier, which is one reason to keep more than one enforcement point.

The web-application check is short:

```
def check_pet_form(data):
    if (data.get("name") or "").strip() == "":
        return "Error: name is required."
    if (data.get("type") or "").strip() == "":
        return "Error: type is required."
    age = (data.get("age") or "").strip()
    if age != "" and not age.isdigit():
        return "Error: age must be a whole number, zero or more."
    owner_id = (data.get("owner_id") or "").strip()
    if owner_id == "":
        return "Error: You must select an owner for the pet."
    if not owner_id.isdigit():
        return "Error: owner_id must be a number."
    return None
```

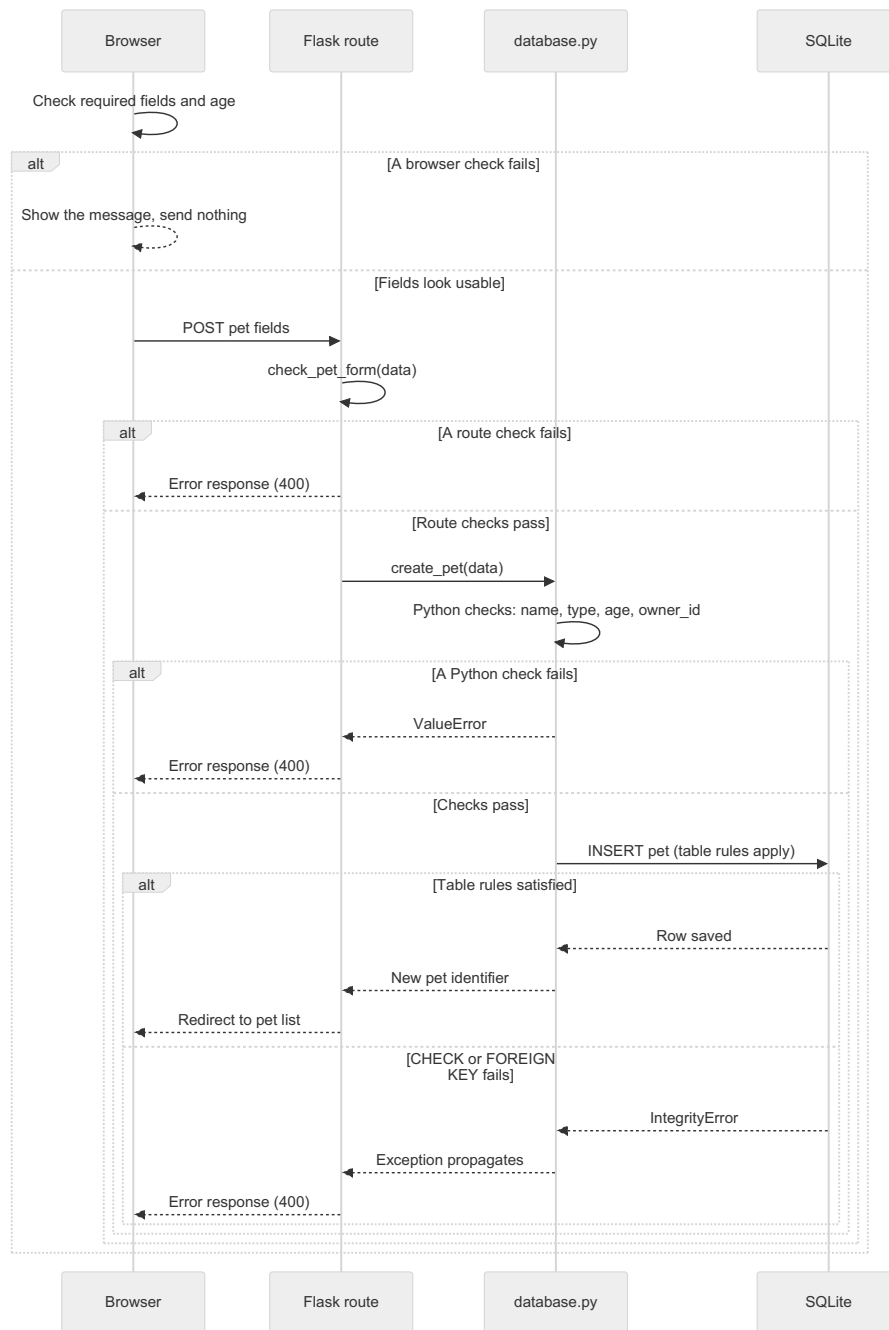
The create and update routes both call it, and return the message with status 400 when it finds a problem. A blank age passes, because a form field left empty is not a mistake. The database layer turns it into zero. The database functions do their own checking before they touch Peewee:

```
def create_pet(data):
    name = (data.get("name") or "").strip()
    pet_type = (data.get("type") or "").strip()
    owner_id = data.get("owner_id")

    if name == "":
        raise ValueError("Pet name is required.")
    if pet_type == "":
        raise ValueError("Pet type is required.")
    if owner_id is None or str(owner_id).strip() == "":
        raise ValueError("owner_id is required.")

    pet = Pet.create(
        name=name,
        type=pet_type,
        age=_normalize_age(data.get("age")),
        owner=int(owner_id),
    )
    return pet.id
```

These repeat the route's checks on purpose. If a rule is wrong in one place, another place still holds it. The price is that a rule now lives in several places, and a change has to reach all of them. The tests are how we find out when one has been missed.



Here is what each enforcement point catches in this example:

Request	Browser	Web application	database.py	SQLite table
Blank pet name or type	required	check_pet_form	create_pet raises ValueError	CHECK
Age of -1	min="0"	check_pet_form	_normalize_age raises ValueError	CHECK
Age of abc	type="number"	check_pet_form	_normalize_age raises ValueError	Not stopped by direct SQL
No owner selected	required	check_pet_form	create_pet raises ValueError	NOT NULL
Owner id 999999	Dropdown offers only real owners	Passes it on	Passes it on	FOREIGN KEY raises IntegrityError
Deleting an owner who has pets	None	Reports the failure	Passes it on	ON DELETE RESTRICT raises IntegrityError

Only the last two rows depend entirely on the table. Whether an owner exists, and whether an owner still has pets, are facts about stored data, and only the layer that holds the data can answer them. Everything else can be checked earlier, and is checked again below.

A Rule Changed

The age rule is different from the earlier examples. In the database-constraints and object-relational-mappers topics, an age that could not be converted quietly became zero. That was tolerable while nothing said an age could be wrong. Now the table says an age must be zero or more, so `_normalize_age` follows suit:

```
def _normalize_age(value):
    if value is None:
        return 0
    if isinstance(value, str) and value.strip() == "":
        return 0

    age = int(value)
    if age < 0:
        raise ValueError("Age must be non-negative.")
    return age
```

A blank age still becomes zero. A negative number is rejected. So is text such as `abc`, since `int("abc")` raises `ValueError` and nothing catches it. Through the web application the route stops that input first, with a plainer message. Direct callers of `create_pet` still get Python's own wording, which is less friendly. Improving it is a reasonable exercise.

Errors Reach the Route as Peewee Exceptions

The earlier version of this application caught `sqlite3.IntegrityError`. Peewee wraps the errors from the database driver in its own exception classes, so the routes now import `IntegrityError` and `OperationalError` from peewee: (2)

```
from peewee import IntegrityError, OperationalError
```

An `IntegrityError` is what you get for a failed CHECK, NOT NULL, or foreign key rule, so the route cannot tell from the class alone which rule fired. The message does. Peewee passes through SQLite's text, such as FOREIGN KEY constraint failed or CHECK constraint failed: age >= 0. The database functions leave these exceptions alone, and each route turns them into a status code and a readable message.

That split follows the layers. The database layer decides which rules exist and refuses to store anything that breaks them. The web application decides what the person at the browser sees. Owner deletion shows it:

```
except IntegrityError as e:
    return error_page(
        "Error: Cannot delete this owner because they have pets. "
        "Please delete their pets first.\n"
        f"(details: {e})",
        400,
    )
```

The route reads a constraint failure as “this owner has pets” because that is the only rule `delete_owner` can break. A route that could trigger several different constraints would need to look harder at the message.

There is a cost to notice. Routes that import Peewee's exception classes now depend on Peewee, which is exactly the dependence the database layer was meant to hide. We could have the database functions catch `IntegrityError` and raise an exception of our own, so the routes never mention Peewee. This example does not, and the tradeoff is worth thinking about.

Reading Pets with Their Owners

A pet row has an owner identifier. The list page wants a name. In the database-constraints topic the route matched identifiers with a loop over two lists. With a mapper you can ask for the join:

```
def get_pets():
    query = Pet.select(Pet, Owner).join(Owner).order_by(Pet.name, Pet.id)
    return [pet_to_dict(pet) for pet in query]
```

`select(Pet, Owner)` names the columns to load from both tables, and `join(Owner)` follows the foreign key. The generated SQL is an ordinary inner join: (5)

```
SELECT "t1"."id", "t1"."name", "t1"."type", "t1"."age", "t1"."owner_id",
       "t2"."id", "t2"."name", "t2"."city", "t2"."type_of_home"
FROM "pet" AS "t1"
INNER JOIN "owner" AS "t2" ON ("t1"."owner_id" = "t2"."id")
ORDER BY "t1"."name", "t1"."id"
```

Then `pet_to_dict` can read `pet.owner.name` without going back to the database, because the owner columns arrived with the pet:

```
def pet_to_dict(pet):
    return {
        "id": pet.id,
        "name": pet.name,
        "type": pet.type,
        "age": pet.age,
        "owner_id": pet.owner_id,
        "owner_name": pet.owner.name,
    }
```

Leave the join out and the code still works, but slowly in a way that is easy to miss. Reading `pet.owner` on a pet whose owner was not loaded runs another query. With three pets and no join, the list took four queries: one for the pets and one per pet. With the join it took one. That pattern has a name, the **N+1 problem**, and Peewee's documentation covers it. (5) The dictionary that leaves the database layer looks the same either way, which is why a test of the output will not reveal the difference.

The `ORDER BY` uses the identifier as a tie-breaker. Two pets with the same name would otherwise come back in whatever order SQLite chose.

Updating and Deleting

`update_pet` loads the pet, changes its attributes, and saves:

```
pet.name = name
pet.type = pet_type
pet.age = _normalize_age(data.get("age"))
pet.owner = int(owner_id)
pet.save()
```

Assigning `int(owner_id)` to `pet.owner` works because Peewee accepts an identifier where it expects an owner. If the identifier names no owner, `save()` raises `IntegrityError` and the stored pet keeps its old values. Moving a pet to another owner is this same assignment. Nothing about either owner record changes.

`delete_owner` fetches the owner and calls `delete_instance()`, which issues a `DELETE` for that row. With `RESTRICT`, SQLite refuses when a pet still refers to it, and the exception reaches the route. Once the last pet is reassigned or deleted, the same call succeeds. Peewee can also delete dependent rows for you, but only when you ask with `recursive=True`, which removes the owner's pets along with the owner. That is exactly the behavior `RESTRICT` was chosen to avoid. (2)

Grouping Writes Around a Constraint

When we first used `atomic()`, we demonstrated a rollback by raising an exception in the middle of a block. Constraints give a more realistic reason to use it. Suppose two pets must be saved together, and the second refers to an owner that does not exist:

```
with database.db.atomic():
    database.create_pet(first_pet)
    database.create_pet(second_pet_with_bad_owner)
```

The second insert raises `IntegrityError`. The exception leaves the block, Peewee rolls the transaction back, and the first pet is not stored either. (10) A test checks this: after the failure, `get_pets()` is empty.

Testing the Rules

There are two test files, and both use temporary databases. `test_database.py` covers the database layer and the upgrade. `test_app.py` sends requests through Flask's test client and covers the web layer.

- Foreign keys are on for the connection.
- A missing owner is rejected and nothing is stored.
- The Python checks reject blank names, blank types, negative and unreadable ages, and a missing owner.
- The `CHECK` constraints reject blank names and types and negative ages when `Pet.create` skips the helper functions.
- An owner with pets cannot be deleted. After the pet is deleted or reassigned, the owner can.
- A failed update leaves the pet unchanged.
- An `atomic()` block rolls back after a constraint failure.
- Blank owner details are stored as `NULL`, and the templates display them as empty text.
- The table definitions contain the rules.
- `initialize` refuses an older file and leaves it untouched.
- The upgrade handles each earlier layout, keeps identifiers, skips rows that break a rule, keeps the original file, and never overwrites an earlier backup.
- The forms carry the browser rules, and the routes reject each kind of bad form with a message and store nothing.

A few of these tests skip an enforcement point on purpose. The test for the CHECK constraints calls `Pet.create` directly, because going through `create_pet` would test the Python checks and never reach the table. The route tests post forms that a browser would have refused. Writing a test that bypasses the check above is how you find out whether the check below is doing its part. The browser layer is the one these tests cannot exercise, since the test client does not run a browser. They check that the attributes are on the form, and you check the behavior by hand.

Practice

Use a working copy of the code directory.

1. Start the application. In the create form, try to submit a blank name, a negative age, and no owner. Note what the browser does. Then open the browser's developer tools, delete the `required` and `min` attributes, and submit again. Which layer answered this time, and how can you tell?
2. Send the same bad requests from Python with the `requests` library or `curl`, so no browser is involved. Compare the responses with the table of layers.
3. Read `sqlite3 pets.db .schema`. Find each rule from the models in the generated SQL.
4. Create an owner with a blank city. Open the owner list and the owner edit form. Explain why the city is empty on both pages.
5. Give one owner two pets. Try deleting the owner, then reassign one pet and try again, then reassign the other and delete the owner.
6. Write a query using `Pet.select()` without the `join`, and count the queries it runs while building the list. Peewee logs each statement through Python's logging module at the `DEBUG` level. Compare with the version that joins.
7. Make a copy of `pets.db` from an earlier example, or build one with that example's setup command. Run `python3 upgrade_database.py` on the copy. Read the report, then compare the tables and the backup file with `sqlite3`.

Exercise: Decide What Makes an Owner Unique

Two owners can share a name, which is why we use an identifier. Suppose the application should still refuse an exact duplicate: the same name and the same city.

Add a unique index on `(name, city)` to the owner model. Peewee lets a model list indexes in its `Meta` class. Then test three cases directly through the database functions: two owners with the same name in different cities, two with the same name and the same city, and two with the same name and no city.

Write down what you expected before you run it. In SQLite, `NULL` values in a unique index count as different from each other, so the third case may not behave the way you would want. Decide whether that is acceptable for this application. Then decide which layers should know about the rule, and what each one should do when it is broken. Remember that a table you change this way is an older table the next time someone opens a database made without it, and `schema_problems` as written would not notice. What would it need to check, or what could the database record, so that it could?

References

All sources accessed September 23, 2026.

1. Charles Leifer. “Models and Fields.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/models.html>
2. Charles Leifer. “Peewee API Documentation.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/api.html>
3. SQLite. “CREATE TABLE,” section on CHECK constraints. https://www.sqlite.org/lang_createtable.html
4. Wikipedia contributors. “Check constraint.” Wikipedia. https://en.wikipedia.org/wiki/Check_constraint
5. Charles Leifer. “Relationships and Joins.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/relationships.html>
6. SQLite. “SQLite Foreign Key Support.” <https://www.sqlite.org/foreignkeys.html>
7. Wikipedia contributors. “Referential integrity.” Wikipedia. https://en.wikipedia.org/wiki/Referential_integrity
8. Charles Leifer. “Database.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/database.html>
9. SQLite. “Datatypes In SQLite,” section on comparison expressions. <https://www.sqlite.org/datatype3.html>
10. Charles Leifer. “Transactions.” Peewee documentation. <https://docs.peewee-orm.com/en/latest/peewee/transactions.html>

Questions for Thought

1. The browser, the route, `database.py`, and the table all reject a negative age. What does each one protect against that the others do not? Which of the four could you remove without any visible change for a person using the browser, and what would you lose?
2. Every rule now lives in several places. What does that cost when a rule changes, and how would you make sure no layer is left behind?
3. When an `IntegrityError` comes back, what does the route know, and what does it have to guess? How would the code change if the same delete could fail for two different reasons?
4. The routes import Peewee’s exception classes. What would it take to hide Peewee from them, and what would you give up?
5. The upgrade skips rows that break the rules and reports them. What else could it do with those rows, and who should decide?
6. A join saves queries, but it also loads owner columns the page may not display. When would you rather not join?
7. Peewee wrote the SQL for these tables. What did you gain by not writing it, and what did you have to check to be sure it said what you meant?